

Web Music Player

Architecture Documentation

structured after the arc42 template (v8)

Nicolà Widmer · HSLU · Web Programming Lab

2026-09-25

Contents

1	Introduction and Goals	3
1.1	Quality Goals	3
1.2	Stakeholders	3
2	Architecture Constraints	4
3	Context and Scope	5
3.1	Business Context	5
3.2	Technical Context	5
3.3	Scope	5
4	Solution Strategy	6
5	Building Block View	7
5.1	Level 1 — System	7
5.2	Level 2 — Backend Module	7
5.3	Level 2 — Frontend	8
6	Runtime View	9
6.1	Play a song	9
6.2	Remove a song from a playlist	9
6.3	Upload a song	9
6.4	Delete a song	10
6.5	Sign in	10
7	Deployment View	11
8	Cross-cutting Concepts	12
8.1	Ownership	12
8.2	Authentication	12
8.3	Consistency between contexts	12
8.4	Frontend	12
8.5	Dependency supply chain	12
8.6	Testing	12
9	Architecture Decisions	14
10	Quality Requirements	15
11	Risks and Technical Debt	17
12	Glossary	18

1 Introduction and Goals

A web music player. Users upload MP3s, group them into playlists and play them on a record-player screen (spinning disc, moving tone-arm). The CRUD resources are **Songs** and **Playlists**. Scope and user stories: `Proposal.typ`.

1.1 Quality Goals

Prio	Goal	Meaning
1	Turntable feel	Disc spins, tone-arm moves. Respects <code>prefers-reduced-motion</code> .
2	Streaming & seeking	Playback starts before the file is downloaded. HTTP Range (206) end to end.
3	Testability	Domain logic runs in unit tests without a database.
4	Privacy	Users only see their own data. Enforced on the server.
5	One-command start	<code>docker compose up</code> on a clean machine.

1.2 Stakeholders

Role	Expects
Developer (Nicolà Widmer)	Clear structure, fast tests.
Lecturer (HSLU)	≥ 2 CRUD resources, tests, Docker, documented decisions.
End user	Reliable upload and playback, private library.

2 Architecture Constraints

Constraint	Consequence
Angular SPA	TypeScript, standalone components.
NestJS REST API	No GraphQL, no SSR.
PostgreSQL	Metadata only. Forward-only migrations (Drizzle).
Audio on the server filesystem; S3 optional	Storage sits behind a port.
Vitest, Supertest, Playwright	Ports must be fakeable. Vitest replaces the proposal's Jest.
Docker Compose	Every dependency is a service; config via env vars.
Evergreen browsers	<audio> and Media Session API assumed.
One developer, fixed deadline	Scope guarded by MoSCoW.

Conventions: DDD + hexagonal backend, one module per bounded context. Decisions are ADRs (`Documentation/adr/`). ESLint + Prettier everywhere.

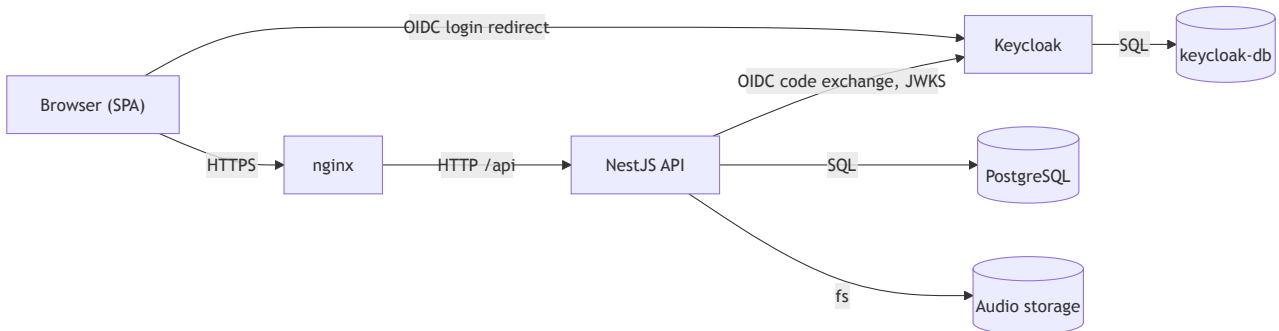
3 Context and Scope

3.1 Business Context



No external services. No email provider: verification and password reset are out of scope.

3.2 Technical Context



3.3 Scope

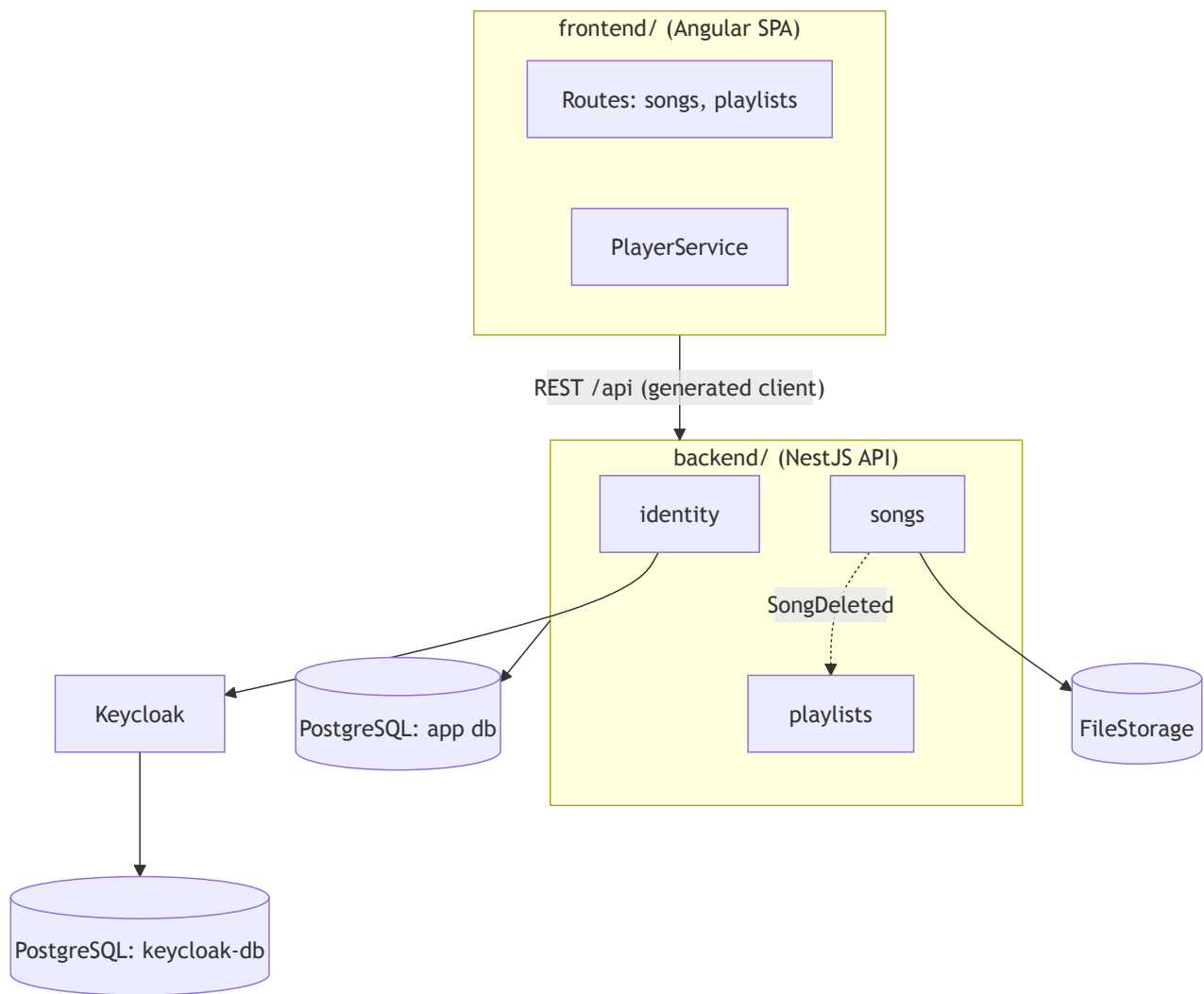
In scope	Out of scope
MP3 upload + ID3 tags, song & playlist CRUD, record-player playback (seek, repeat, volume, resume), range streaming, Keycloak login, per-user data.	Email verification, password reset, non-MP3 formats, sharing, native apps, horizontal scaling.

4 Solution Strategy

Goal	Approach	ADR
Simplicity	One repo, one backend process.	0001-monorepo-monolith
Testability	DDD + hexagonal NestJS modules: <code>identity</code> , <code>songs</code> , <code>playlists</code> . Contexts talk via <code>async in-process events</code> .	0002-ddd-hexagonal-backend
Streaming, one origin	nginx serves the SPA and proxies / <code>api</code> .	0003-nginx-serves-frontend
Portability	Metadata in PostgreSQL, audio behind a <code>FileStorage</code> port.	0004-metadata-postgres-blob-storage-port
Privacy	Keycloak login, session cookie, every query scoped by owner.	0005-session-cookie-auth
One contract	OpenAPI generated from the backend, typed client generated from that. TanStack Query for server state.	0006-openapi-typed-client-tanstack-query
Maintainable UI	Presentational/container split; playback in one service.	0007-frontend-component-communication
Turntable feel	CSS animation gated on <code>prefers-reduced-motion</code> .	—

5 Building Block View

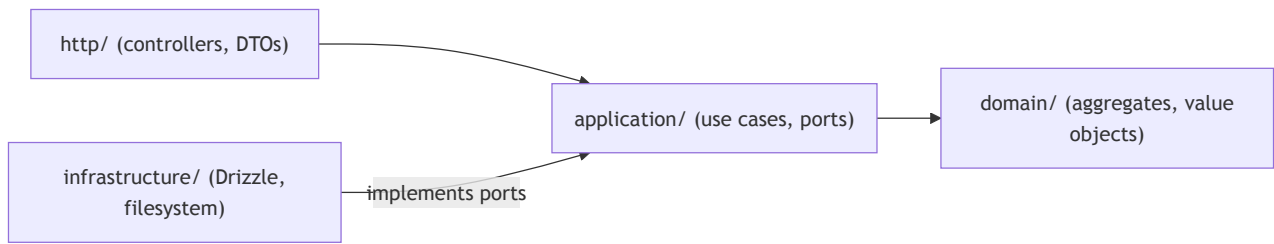
5.1 Level 1 — System



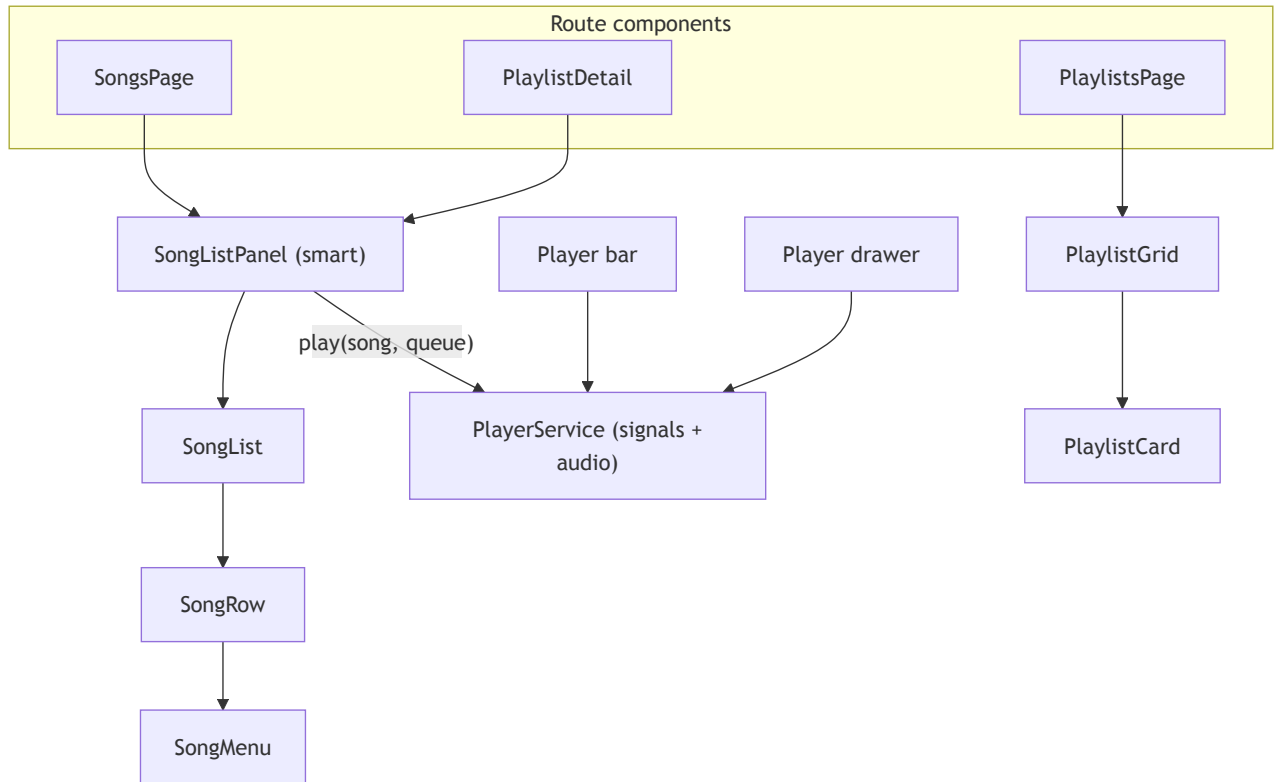
Block	Responsibility
frontend/	UI and client-side playback state. No business rules.
backend/	All rules and owner checks. Detail: <code>domain-model.typ</code> .
nginx	Serves the SPA, proxies <code>/api</code> , terminates TLS.
PostgreSQL db	App data: songs, playlists, users, sessions.
FileStorage	Audio and cover bytes. Local filesystem.
Keycloak	Login and registration (OIDC).
PostgreSQL keycloak-db	Keycloak's own data. The API never touches it.

5.2 Level 2 — Backend Module

Every context has the same four layers. Dependencies point inward.



5.3 Level 2 — Frontend



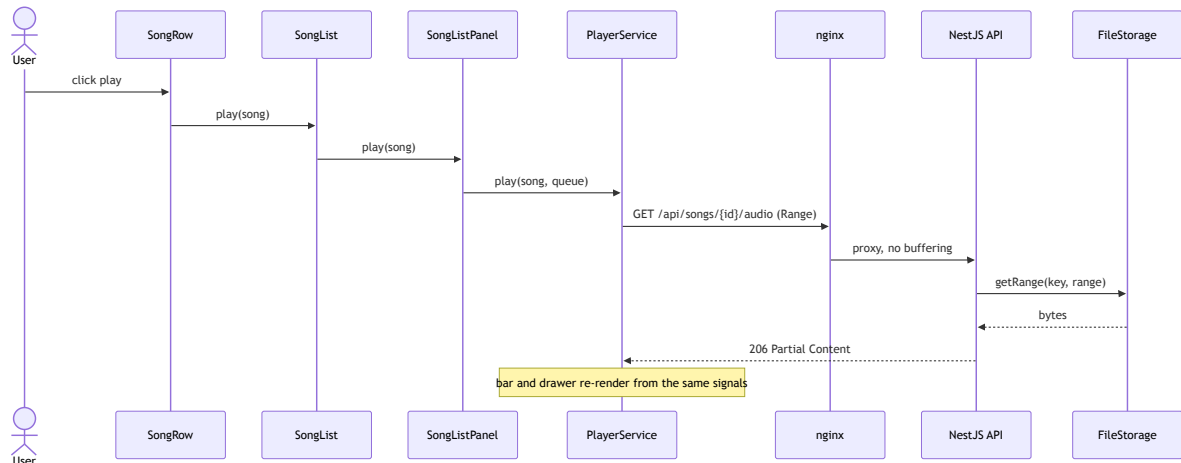
Folder	Contents
api/	Generated client + query options. Don't edit.
auth/	Login redirects, 401 handling, user menu.
shared/songs/	Song list panel and its presentational parts.
features/songs/	Songs route, upload.
features/playlists/	Playlists routes, grid, cards, dialogs.
features/player/	PlayerService, bar, drawer, turntable UI.

State has two homes (0006-openapi-typed-client-tanstack-query):

- **Server state** lives in TanStack Query. Never copied into signals.
- **Client state** lives in a component signal, or in **PlayerService** when several components need it at the same time.

6 Runtime View

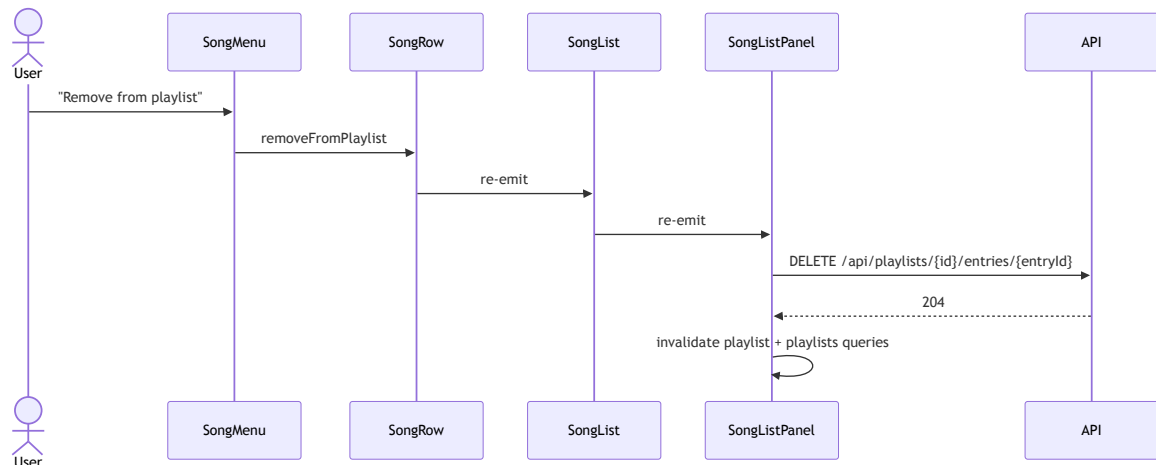
6.1 Play a song



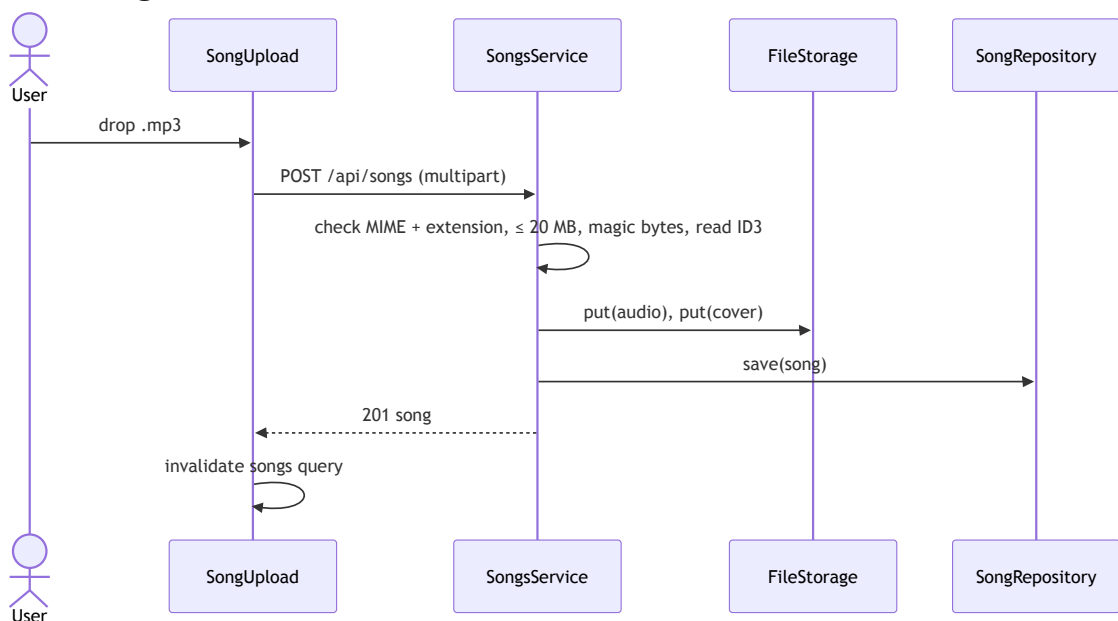
SongList only forwards the event. SongListPanel adds the queue (the rows in display order), which only it knows. PlayerService keeps a copy of that queue, so opening another page doesn't change what plays next. See 0007-frontend-component-communication for why rows don't call the service directly.

6.2 Remove a song from a playlist

The deepest event chain: one emit, two unchanged re-emits, one owner.

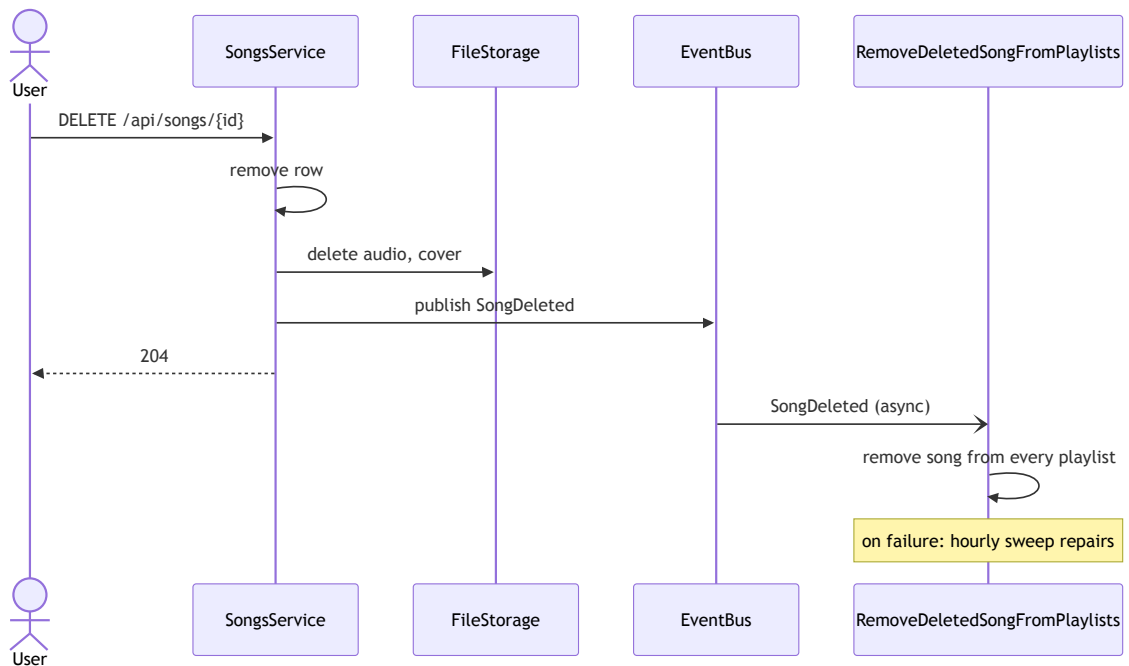


6.3 Upload a song

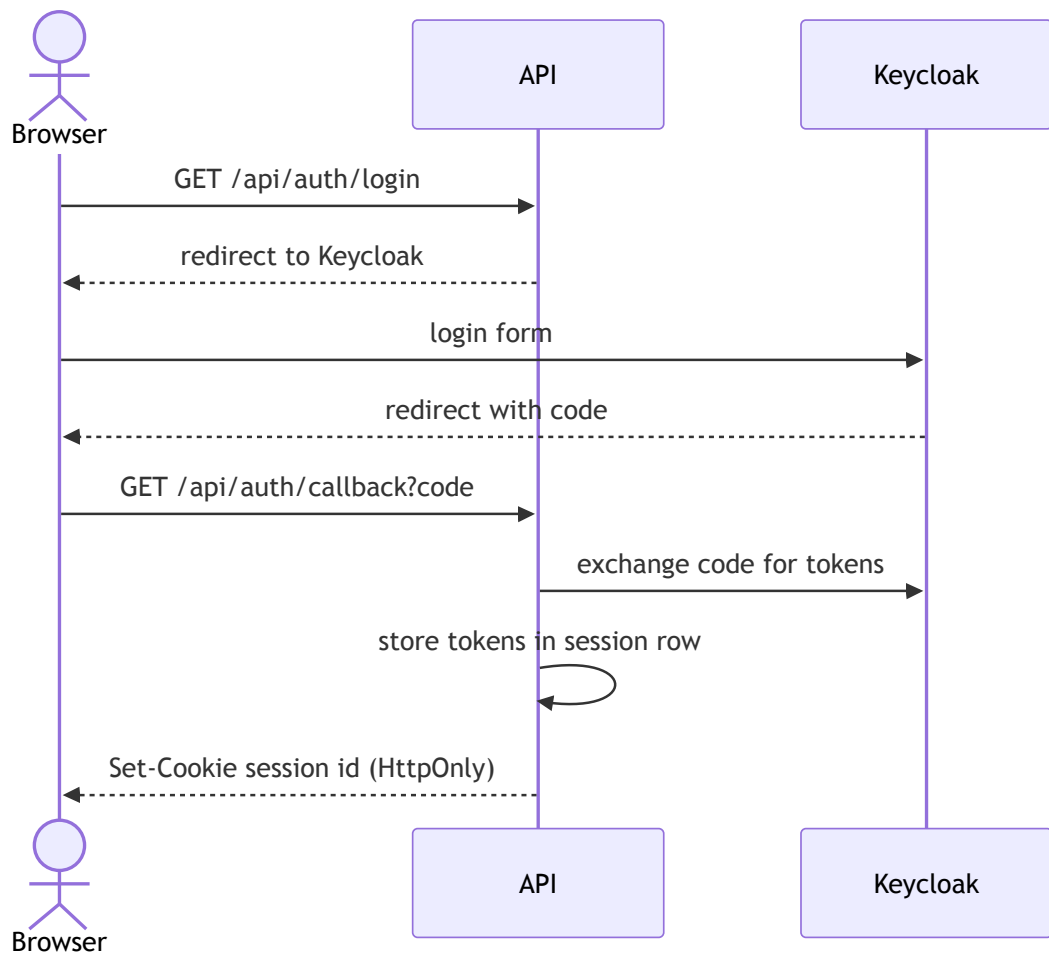


6.4 Delete a song

The response returns before playlists are cleaned up.

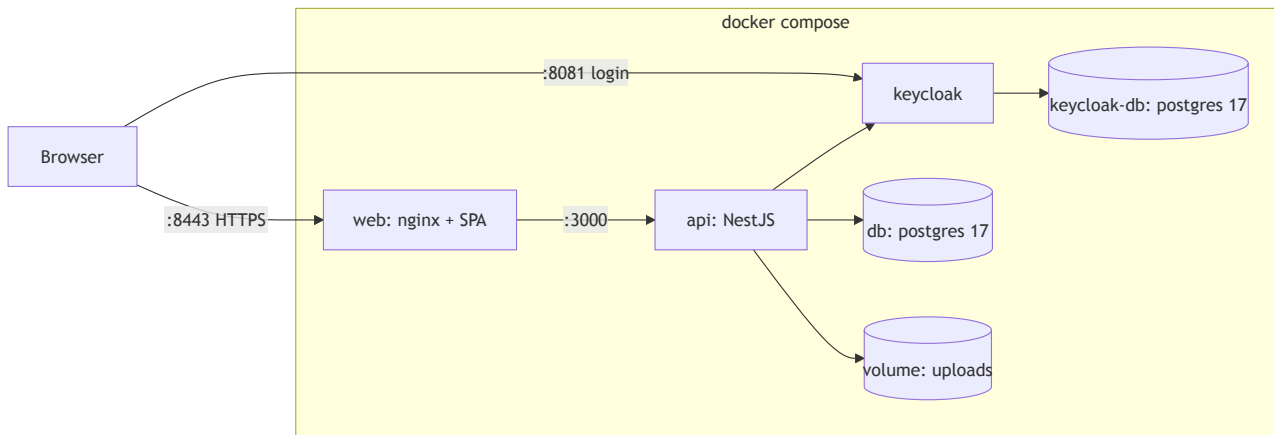


6.5 Sign in



7 Deployment View

One Docker Compose stack (`docker compose up`).



Service	Notes
web	nginx with the built SPA. Ports 8080 / 8443. Self-signed cert.
api	Runs migrations on start. Only reachable through web.
db	App database. Volume db-data .
keycloak	Imports the realm from keycloak/import/ on first start.
keycloak-db	Keycloak's own database.
minio	Only with <code>--profile storage-s3</code> . Not used by the API yet.

Native dev: `pnpm dev` runs Postgres in Docker and the API and SPA with hot reload (`mprocs`). SPA on `:4200`.

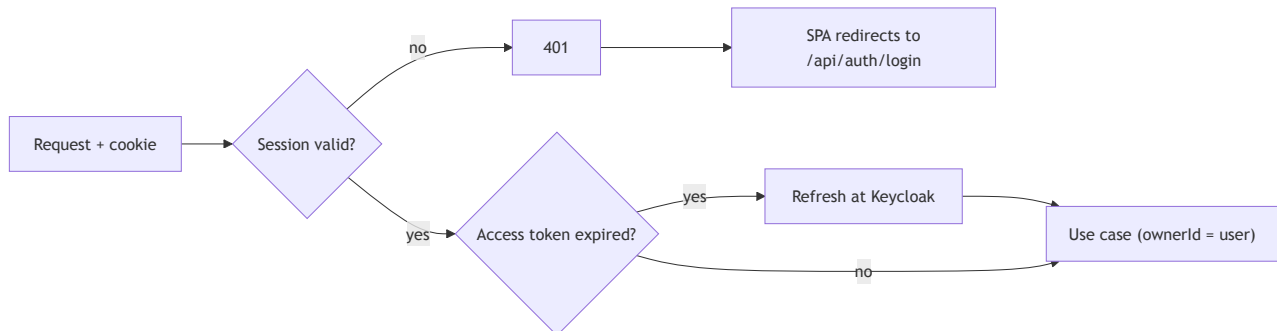
8 Cross-cutting Concepts

8.1 Ownership

Every repository read takes an `ownerId`. Another user's resource returns 404 (0005-session-cookie-auth).

8.2 Authentication

The SPA never sees a token. It only sends an `HttpOnly` session cookie.



`AUTH_ENABLED=false` skips all of this and uses one fixed local user.

8.3 Consistency between contexts

`SongDeleted` is the only cross-context event. It is fire-and-forget. An hourly sweep removes playlist entries whose song no longer exists. Another hourly job deletes expired sessions.

8.4 Frontend

Topic	Approach
Events	Leaves emit; at most two re-emits; one smart component owns the mutation (0007-frontend-component-communication).
Forms	Angular Signal Forms. Server validates again.
i18n	@ngx-translate, all strings in <code>public/i18n/en.json</code> .
Styling	Tailwind 4 + Optimus UI. Dark mode follows the OS.
Motion	Turntable animation off under <code>prefers-reduced-motion</code> .
Accessibility	WCAG 2 AA checked by <code>pnpm ally</code> (<code>pally-ci</code>) on the live stack.

8.5 Dependency supply chain

`pnpm` only installs versions that were published at least **5 days** ago (`minimumReleaseAge: 7200` minutes in the root, `backend/` and `frontend/` `pnpm-workspace.yaml`). `pnpm install --frozen-lockfile` also rejects a committed lockfile that contains a younger version.

Pros	Cons
Most malicious npm releases (hijacked maintainer accounts, worm-style self-publishing) are found and unpublished within hours to a few days. A 5-day delay means we never install them.	Security fixes also arrive 5 days late. An urgent patch needs a manual exception (<code>minimumReleaseAgeExclude</code>) or a temporary override.
No extra tool or service. The check runs inside <code>pnpm</code> , in local installs and in the Docker builds.	Freshly released versions cannot be used right away. A new package added in its first days fails to resolve.
Upgrades are calmer: broken releases are usually fixed or deprecated before we pick them up.	It only protects against <i>new</i> malicious versions. An old compromised version or a malicious package that stays unnoticed longer is not caught.

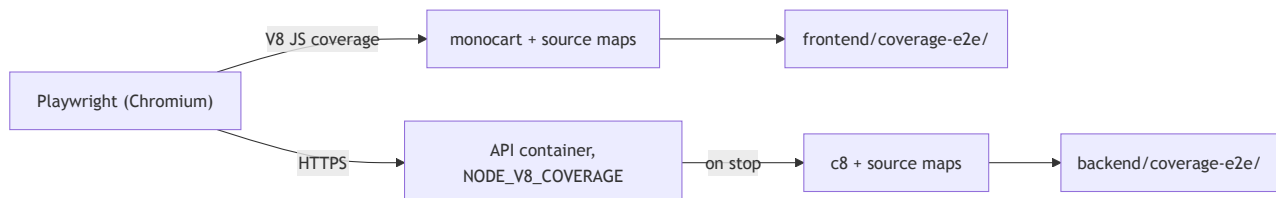
8.6 Testing



Type	Tool	Count	What it covers
Backend unit	Vitest	35 tests, 6 files	Auth config, session guard, token verifier, session sweep, file storage, ID3 reader. Fake ports.
Backend integration	Supertest + Testcontainers	37 tests, 5 files	HTTP endpoints against a real Postgres: songs, playlists, identity, user provisioning, the <code>SongDeleted</code> flow.
Frontend unit	Vitest (ng test)	2 tests, 1 file	App shell renders.
E2E	Playwright	8 tests, 5 files	Login redirect, navigation, upload/edit/delete song, reject non-MP3, playlist lifecycle, playback, song menu (add to playlist twice, Move Up/Down, Play), deleting a song removes it from playlists. Full Compose stack + real Keycloak.
Accessibility	pally-ci	3 pages	WCAG 2 AA on songs, playlists, playlist detail.

Coverage (2026-09-23)	Lines	Statements	Branches	Functions
Backend — unit + integration	85.8 %	85.8 %	87.5 %	84.1 %
Backend — E2E	85.7 %	85.7 %	68.9 %	83.0 %
Frontend — unit	30.4 %	29.5 %	34.1 %	14.8 %
Frontend — E2E	82.9 %	78.2 %	56.7 %	71.2 %

Scope: hand-written code in `backend/src/` and `frontend/src/app/`. Not counted: the generated `api/` client, specs, templates. Files that no test loads count as 0 %. The rows are not merged.



Command	Produces
<code>pnpm test:coverage</code> in <code>backend/</code> or <code>frontend/</code>	Unit (+ integration) coverage in <code>coverage/</code> .
<code>pnpm e2e:coverage</code> at the repo root	Rebuilds the stack with source maps and coverage recording, runs Playwright, writes both <code>coverage-e2e/</code> reports, then restores the normal stack.

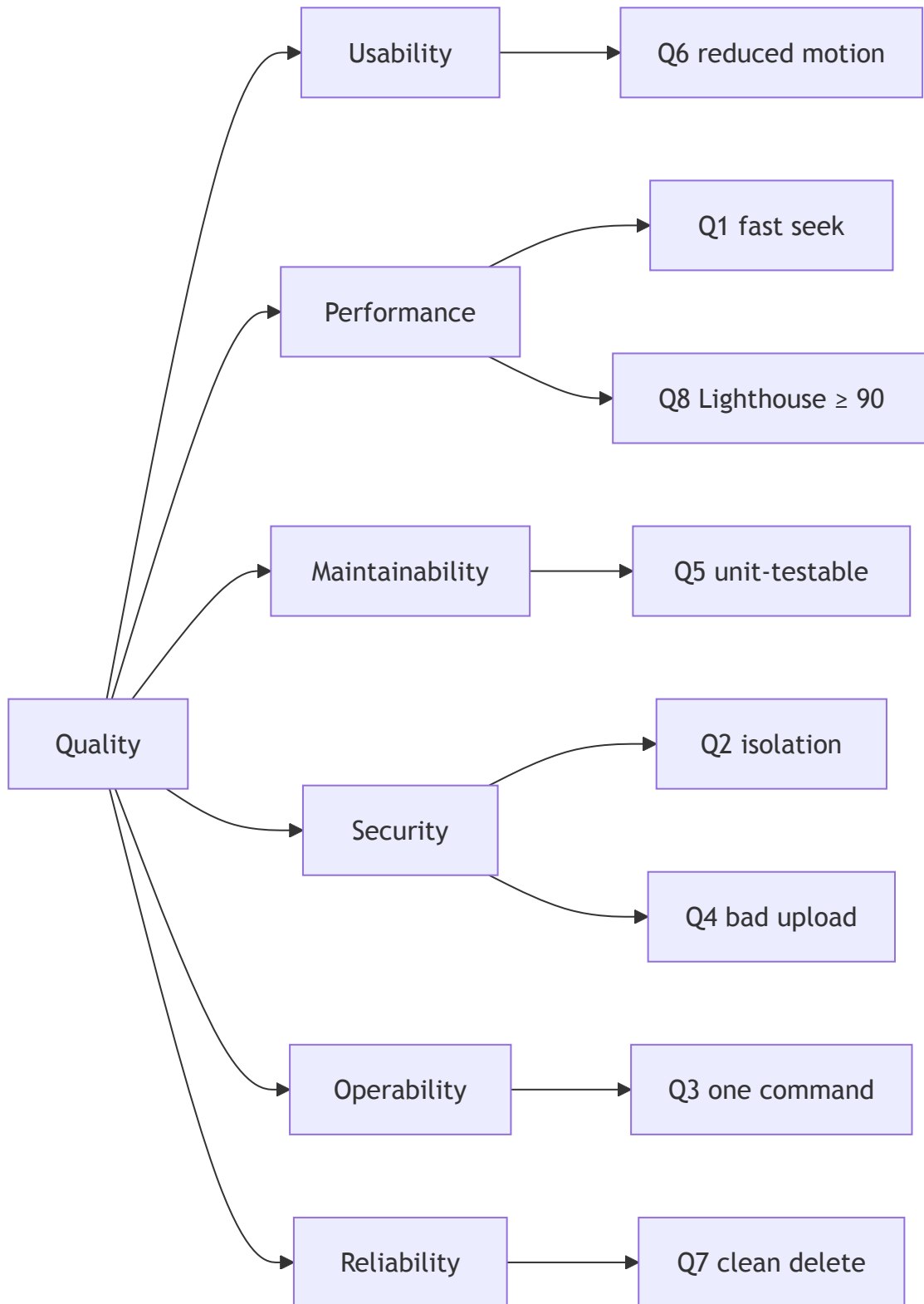
The coverage build turns off Angular’s optimizer, otherwise inlined code can’t be mapped back to its source file. The Playwright fixture saves coverage before every `goto/reload`, because a full page load discards the old counts. Backend numbers are high even for E2E: Node counts code that runs at startup (module loading, DI wiring) as covered.

9 Architecture Decisions

One ADR per decision in `Documentation/adr/`.

ADR	Decision
0001-monorepo-monolith	One repo, one backend process.
0002-ddd-hexagonal-backend	DDD + hexagonal modules, async in-process event bus.
0003-nginx-serves-frontend	nginx serves the SPA and proxies <code>/api</code> .
0004-metadata-postgres-blob-storage-port	PostgreSQL for metadata, audio behind a <code>FileStorage</code> port.
0005-session-cookie-auth	Keycloak (OIDC), BFF session cookie, owner-scoped queries.
0006-openapi-typed-client-tanstack-query	Generated OpenAPI client, TanStack Query.
0007-frontend-component-communication	Presentational/container split, max two re-emits.

10 Quality Requirements



ID	Scenario	Expected
Q1	User seeks on a slow mobile connection.	Plays from new position in < 1 s with one 206 request.
Q2	User A requests B's song.	404.
Q3	docker compose up on a machine with only Docker.	App on :8443, migrations applied, no manual step.

Q4	Upload of a 25 MB file, a non-MP3, or a non-MP3 renamed to .mp3.	400; nothing stored.
Q5	New playlists use case.	Unit-testable with fake ports.
Q6	OS “reduce motion” is on.	Disc does not spin; audio works.
Q7	Delete a playing song that is in 3 playlists.	Row and file gone, entries removed, player advances.
Q8	Signed-in user opens the prod build (docker compose up, :8443); Lighthouse 12, mobile and desktop presets.	Performance, Accessibility, Best Practices and SEO each ≥ 90 .

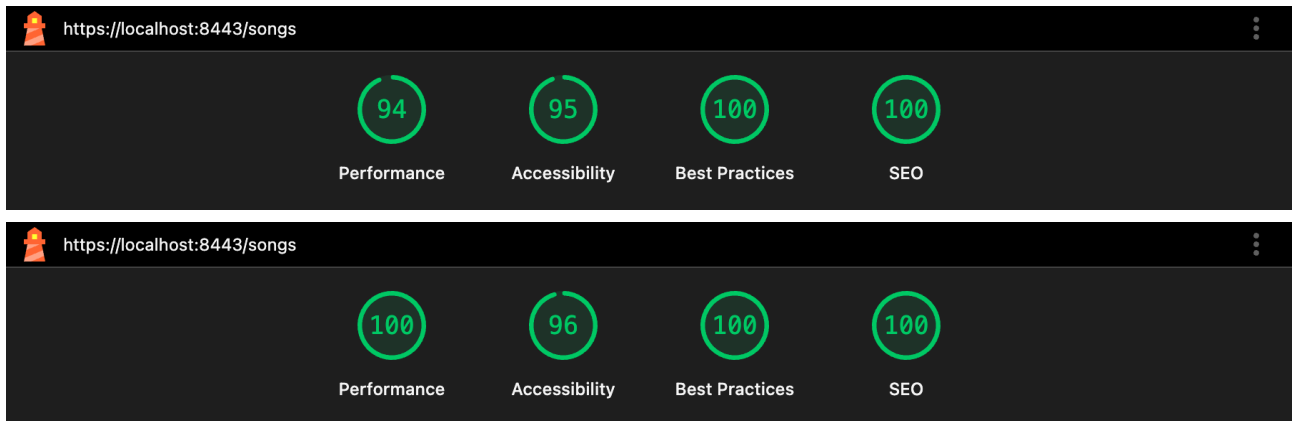


Figure 1: Q8 evidence: Lighthouse 12.8 on /songs, mobile (top: 94/95/100/100) and desktop (bottom: 100/96/100/100), measured 23.09.2026.

11 Risks and Technical Debt

Area	Problem	Status
E2E / a11y isolation	Tests share one user and one database. A failed test can leave data behind.	Tests run serially and clean up. Fine for one developer.
Event-chain depth	The two-re-emit rule (ADR-0007) is not linted.	Checked in review.
pa11y login	Each scanned URL logs in again.	Fine for 3 URLs.
Bundle size	Initial bundle over Angular's 1 MB warning.	Low priority: self-hosted.
S3 storage	Adapter not built; only local filesystem.	Optional (Could).
Logging and telemetry	Only a few places log, via NestJS's default console <code>Logger</code> (OIDC controller, session sweeper, playlist sweep). No structured (JSON) logs, no request or correlation IDs, and nothing logs the async domain events. No metrics, tracing or health endpoint. The frontend reports no errors.	Open. Failures in production show up only in container stdout. A fix would add structured logging (e.g. pino), a health check and OpenTelemetry.
Dev credentials	The compose stack is dev-only. The Keycloak realm import seeds user <code>dev/dev</code> and client secret <code>dev-secret-change-me</code> . Keycloak runs <code>start-dev</code> with admin <code>admin/admin</code> and self-registration on. Postgres (<code>wmp</code>) and MinIO (<code>minioadmin</code>) also fall back to default credentials.	Accepted for local use. Before any deployment: separate realm without seeded users, generated client secret, <code>start</code> mode, and secrets from env vars without <code>:-default</code> fallbacks.

12 Glossary

Term	Meaning
Aggregate	Objects changed as one unit. The root guards the rules.
Value object	Immutable, compared by value (e.g. <code>Email</code>).
Bounded context	Area with one model: <code>identity</code> , <code>songs</code> , <code>playlists</code> .
Port / adapter	Interface the core defines / its implementation.
Domain event	Fact published to other contexts (<code>SongDeleted</code>).
BFF	Backend-for-frontend: the API holds the tokens, the browser a cookie.
OIDC	Login protocol used with Keycloak.
Range / 206	Fetch part of a file. Enables seeking.
ID3	Tags inside an MP3 (title, artist, cover).
ADR	Architecture Decision Record.
MoSCoW	Must / Should / Could / Won't.